Electronics & Electrical Communications Department

Introduction to Communications - EECG232

Software Radio
Random Process Analysis

By

Team 11

Submitted to

Dr. Mohamed Nafi **Eng. Mohamed Khaled**

April 30, 2026

Contents

1 Introduction 1
2 Control Flags 1
3 Generation of data 2
3.1 Unipolar Signaling 2
3.2 Polar NRZ Signaling 2
3.3 Polar RZ Signaling 2
3.4 Ensemble Generation 3
3.5 visualizing Waveforms 5
4 Analysis 6
4.1 The Statistical Mean 6
4.2 The Ensemble Autocorrelation Function 7
4.3 The Time Mean and Autocorrelation of One Waveform 9
4.4 Bandwidth of the Transmitted Signal 12
Bibliography 21

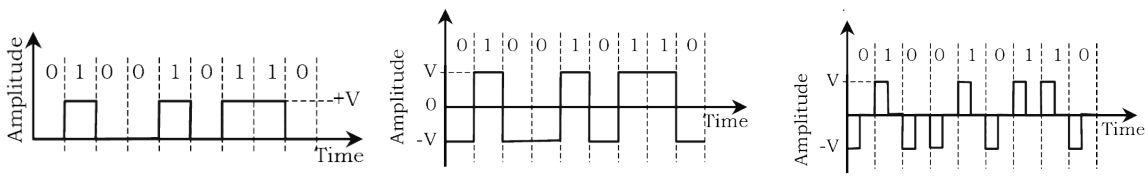
1 Introduction

Software Radio is a technique wherein digital signal processing is used to perform radio communication functions traditionally implemented in analog hardware. This approach allows for greater flexibility and adaptability in communication systems, enabling the implementation of various modulation schemes, error correction techniques, and signal processing algorithms through software updates rather than hardware changes.

This project focuses on generating line encodings for randomly transmitted bits and analyzing the statistical properties of the resulting waveforms. The statistical analysis serves a practical engineering purpose as an ideal transmitted signal should form a stationary and ergodic random process.

Stationarity ensures that the signal's spectral density (PSD) is stable over time, enabling the definition of a fixed power spectral density and predictable bandwidth occupancy. Ergodicity ensures that a single observed waveform is statistically representative of the entire ensemble, which is critical in practice since it can greatly reduce costs by requiring fewer testing procedures during design validation and characterization. Together these properties form the theoretical foundation for receiver filter design, bandwidth estimation, and BER analysis.

This project explores 3 different encoding techniques. Figure 1 shows methods for representing the same binary data in a discrete method. The properties of each will be explored.



a) Unipolar Signaling b) Polar NRZ Signaling c) Polar RZ Signaling

Figure 1: Different line encodings for the same bit sequence

2 Control Flags

To easily configure and manage the entire simulation from a single centralized location, the following control parameters are defined at the very beginning of the script. They govern the data generation process: the signal amplitude (`A = 4`), the resolution of each bit (`sample_per_bit = 8`), and the physical time duration of a single sample (`sample_period = 10e-3`). Furthermore, the dimensions of the random process are defined by the length of each individual waveform (`n_bits = 100`) and the total number of simulated realizations within the ensemble (`n_waveforms = 500`).

```
% Control Flags (Defaults)
control_flags.A = 4;
control_flags.sample_per_bit = 8;
control_flags.sample_period = 10e-3;
control_flags.n_bits = 100;
control_flags.n_waveforms = 500;
```

3 Generation of data

The following MATLAB code generates a random bit stream of 100 bits:

```
random_bit_stream = randi([0 1], 1, 100);
```

The `randi([0 1], x, y)` function returns an $x \times y$ array of uniformly distributed numbers between 0 and 1.

3.1 Unipolar Signaling

For unipolar signaling, the bits `[0 1]` map to `[0 A]` to get the logic levels. The Repeat Matrix function `repmat(levels, spb, 1)` duplicates the $1 \times$ `n_bits` matrix (`spb`) times vertically creating a `spb` $\times$ `n_bit` matrix. The `reshape()` function transforms the matrix into a $1 \times$(`spb` $\cdot$ `n_bits` list along the columns:

A visualized example for `n_bit = 2` and `spb = 2`

$$\text{bits: } [0\ 1] \xrightarrow{\times \mathrm{A}} [0\ \mathrm{A}] \xrightarrow{\text{repmat}} \begin{bmatrix} 0 & \mathrm{A} \\ 0 & \mathrm{A} \end{bmatrix} \xrightarrow{\text{reshape}} [0\ 0\ \mathrm{A}\ \mathrm{A}]$$

```
function waveform = unipolar_nrz(bits, A, spb)
  levels = bits * A;
  waveform = reshape(repmat(levels(:)', spb, 1), 1, []);
end
```

3.2 Polar NRZ Signaling

The mapping for polar non-return-to-zero maps `[0 1]` into `[-A A]` through the following transformations:

$$\text{bits: } [0\ 1] \xrightarrow{\times 2} [0\ 2] \xrightarrow{-1} [-1\ 1] \xrightarrow{\times \mathrm{A}} [-\mathrm{A}\ \mathrm{A}]$$

MATLAB/Octave code for Polar NRZ:

```
function waveform = unipolar_nrz(bits, A, spb)
  levels = bits * A;
  waveform = reshape(repmat(levels(:)', spb, 1), 1, []);
end
```

3.3 Polar RZ Signaling

For polar return-to-zero, we first initialize the waveform matrix size and only set half of the `spb` to `A` and `-A` for the high and low bits respectively:

A visualized example if `n_bit = 2` and `spb = 4`

$$[0\ 1] \xrightarrow{\times 2} [0\ 2] \xrightarrow{-1} [-1\ 1] \xrightarrow{\times \mathrm{A}} [-\mathrm{A}\ \mathrm{A}] \xrightarrow{\text{repmat}} \begin{bmatrix} -\mathrm{A} & \mathrm{A} \\ -\mathrm{A} & \mathrm{A} \\ -\mathrm{A} & \mathrm{A} \\ -\mathrm{A} & \mathrm{A} \end{bmatrix} \xrightarrow{\text{Zero}} \begin{bmatrix} -\mathrm{A} & \mathrm{A} \\ -\mathrm{A} & \mathrm{A} \\ 0 & 0 \\ 0 & 0 \end{bmatrix} \xrightarrow{\text{reshape}} [-\mathrm{A}\ -\mathrm{A}\ 0\ 0\ \mathrm{A}\ \mathrm{A}\ 0\ 0]$$

```
function waveform = polar_rz(bits, A, spb)
  half_spb = floor(spb / 2);
  levels = (2*bits(:)' - 1) * A;
  raw_matrix = repmat(levels, spb, 1);

  % Zero out the second half of each bit period
  raw_matrix(half_spb+1:end, :) = 0;

  waveform = reshape(raw_matrix, 1, []);
end
```

3.4 Ensemble Generation

The Random Process is characterized by randomized bits and a phase shift implemented as a circular shift of a random integer between 0 and `spb-1`.

For a given random process $x(t)$, an experiment generates a **Realization** for the process. Figure 2 showcases an interactive polar NRZ random process which can be tried Here. Clicking the shuffle button generates new realizations.

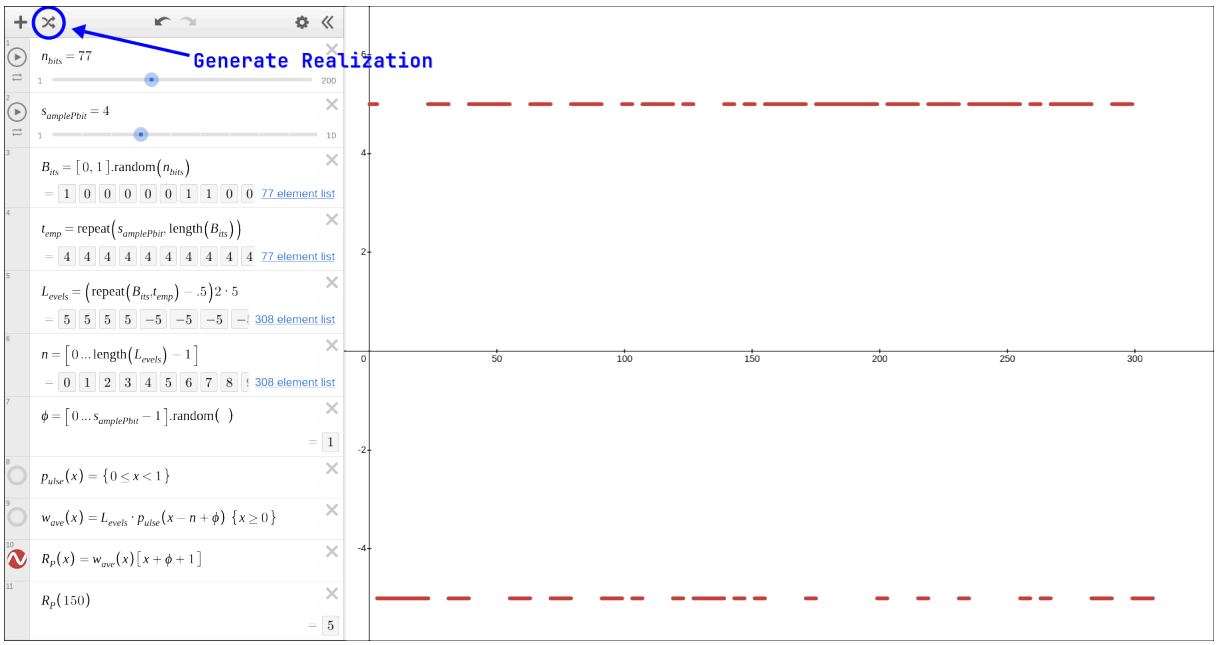


Figure 2: Polar NRZ Random Process.

Multiple such realizations can be simulated to construct an **Ensemble** of random variables that can be used to calculate the statistical properties of the random process.

`generate_ensemble()` acts as the core engine for generating a random process, using a flexible input system where `nargin` and `isempty` checks and prioritize user-provided arguments over the default values stored in `control_flags`.

To ensure high performance, it pre-allocates the ensemble matrix with `zeros(n_waveforms, waveform_length)`.

Inside the for loop, it generates a unique `random_bit_stream` for every realization and passes it to the `line_code_func` handle, allowing it to produce waveforms based on the desired line code.

To model a WSS random process, the code calculates a phase variable ranging from `0` to `spb-1`. It then applies this offset using `circshift(raw_wf, [0, -phase])`, which ensures that the bit transitions do not always align perfectly with the start of the sample buffer. Each shifted `raw_wf` is then stored as a row in the final matrix, creating a comprehensive ensemble ready for statistical analysis.

```
function ensemble = generate_ensemble(line_code_func, control_flags,
n_waveforms, n_bits, A, spb)
  % Use provided arguments or fall back to control_flags struct
  if nargin < 6 || isempty(spb), spb = control_flags.sample_per_bit;
end
  if nargin < 5 || isempty(A), A = control_flags.A; end
  if nargin < 4 || isempty(n_bits), n_bits = control_flags.n_bits; end
  if nargin < 3 || isempty(n_waveforms), n_waveforms =
control_flags.n_waveforms; end

  % Direct allocation for memory efficiency
  waveform_length = n_bits * spb;
  ensemble = zeros(n_waveforms, waveform_length);

  for i = 1:n_waveforms
      random_bit_stream = randi([0 1], 1, n_bits);
      raw_wf = line_code_func(random_bit_stream, A, spb);

      % Apply random phase by circular shift and store directly in
matrix
      phase = randi([0, spb-1]);
      ensemble(i, :) = circshift(raw_wf, [0, -phase]);
  end
end
```

To make things cleaner, a function called `generate_all_ensembles()` is implemented to return a cell array containing 3 ensembles, one for each line code.

```
function ensembles = generate_all_ensembles(control_flags, n_waveforms,
n_bits, A, spb)
    % Pass along optional arguments (empty brackets will trigger
defaults in generate_ensemble)
    if nargin < 5, spb = []; end
    if nargin < 4, A = []; end
    if nargin < 3, n_bits = []; end
    if nargin < 2, n_waveforms = []; end

    % Generates ensembles in order: Unipolar, Polar NRZ, Polar RZ
    ens_uz = generate_ensemble(@unipolar_nrz, control_flags,
n_waveforms, n_bits, A, spb);
    ens_pz = generate_ensemble(@polar_nrz, control_flags, n_waveforms,
n_bits, A, spb);
    ens_rz = generate_ensemble(@polar_rz, control_flags, n_waveforms,
n_bits, A, spb);
```

```
    ensembles = {ens_uz, ens_pz, ens_rz};
end
```

3.5 visualizing Waveforms

`plot_sample_waveforms()` displays the first five waveforms of each ensemble. its implementation is not shown here to keep the focus on the core of simulation logic, though it is available in the full code at the end of the report.

```

%% Generate Ensembles
ensembles_5h = generate_all_ensembles(control_flags);

%% Plot waveforms
plot_sample_waveforms(ensembles_5h, control_flags);

```

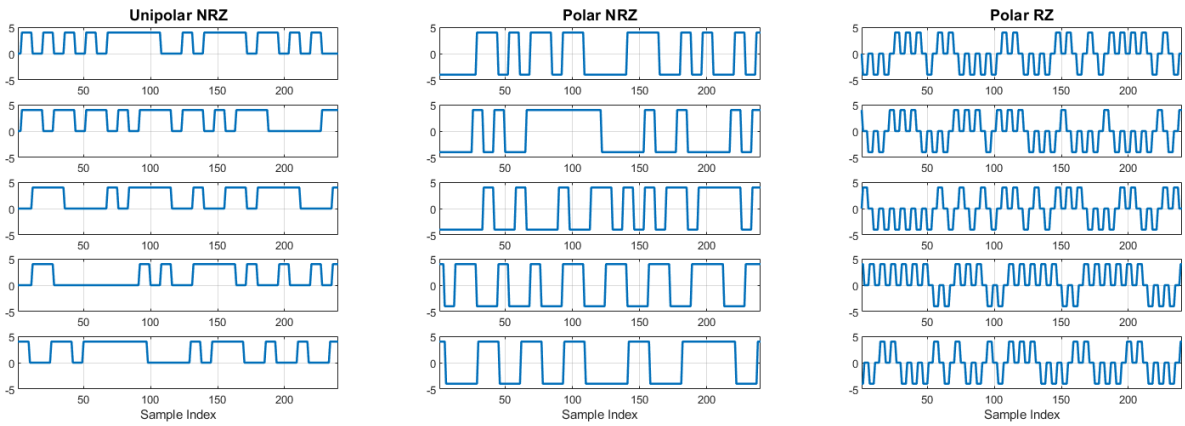


Figure 3: Realizations for each type of line encoding.

The waveforms in Figure 3 were generated using the above code, showing a sample of the structure of the three line codes.

4 Analysis

The following analysis is performed on an ensemble of $N = 500$ waveforms, each containing $L = 100$ bits at spb $= 8$ samples per bit, giving a total waveform length of $L_s = 800$ samples. The amplitude is $A = 4$.

4.1 The Statistical Mean

The statistical mean is the expected value of the random process at each time instant t, averaged across all N realizations:

$$m_x(t) = E[x(t)] = \frac{1}{N}\sum_{i=1}^{N} x_{i(t)} \quad [1]$$

This yields a 1×800 vector — one mean value per time sample.

A function called `plot_means()` was implemented. It takes a cell array of ensembles then calculates the mean and plots it for every ensemble

```

function plot_means(ensembles)
  names = {'Unipolar NRZ', 'Polar NRZ', 'Polar RZ'};
  colors = {'k', 'b', 'r'};
  n_waveforms = size(ensembles{1}, 1);

  figure; hold on;
  for i = 1:length(ensembles)
    current_ens = ensembles{i};
    ens_mean = sum(current_ens, 1)/size(current_ens, 1);

    average = sum(ens_mean)/length(ens_mean);
    variance = var(ens_mean);

    plot(ens_mean, 'Color', colors{i},...
      'LineWidth', 1.5, 'DisplayName', names{i});
    stats_text = sprintf('%s: \\mu = %.4f, \\sigma^2 = %.4e', names{i},
average, variance);
    text(0.02, 0.98 - (i-1)*0.06,...
      stats_text, 'Units', 'normalized', ...
        'Color', colors{i}, 'FontSize', 10,...
          'FontWeight', 'bold', 'VerticalAlignment', 'top');
  end
  ylim([-1, 4]); xlabel('Samples'); ylabel('Ensemble Mean Amplitude');
  title(sprintf('Ensemble Means and Variance (%d waveforms)',
n_waveforms));
  legend('Location', 'northeast'); grid on; hold off;
end
```

we then run the following code to see the results

```
ens_5h = generate_all_ensembles(500);
plot_means(ens_5h)

ens_5k = generate_all_ensembles(5000);
plot_means(ens_5k)
```

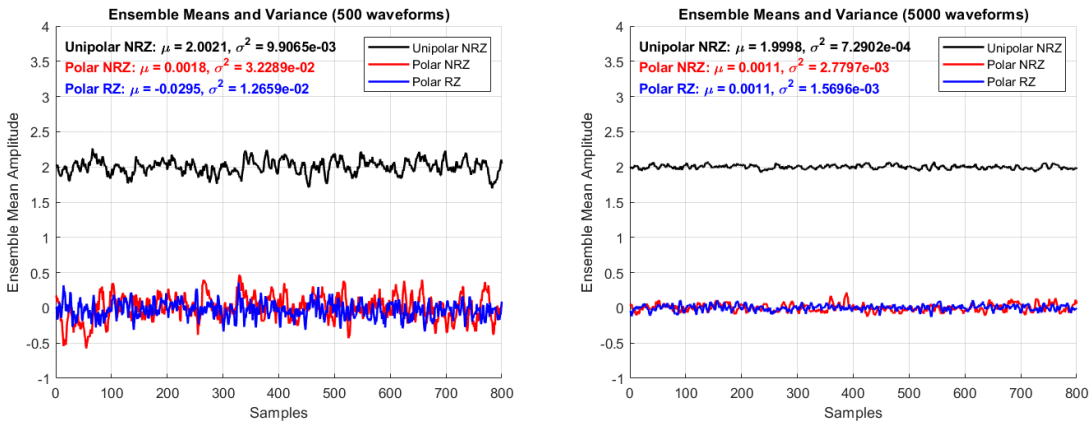


Figure 4: The statistical mean of each line-encoding for 500 and 5k waveforms.

Figure 4 shows how the mean is **nearly constant** across time with a smaller variance over a larger ensemble.

4.2 The Ensemble Autocorrelation Function

The ensemble autocorrelation function says how much a certain reading influences the next one separated by a lag τ:

$$R_x(t,\tau) = E[x(t)\cdot x(t+\tau)] = \frac{1}{N}\sum_{i=1}^{N}\frac{1}{L_s-\tau}\sum_{t=1}^{L_s-\tau} x_i(t)\cdot x_i(t+\tau) \quad [2]$$

The function `ensemble_autocorr` calculates the autocorrelation function (R_x) for a signal ensemble by averaging across both time and all realizations (the statistical ensemble). First, It isolates a portion of the ensemble starting from t_start to the end to focus the analysis on a specific time window. Then, It iterates through a range of time offsets from -max_lag to +max_lag. For each lag k, it multiplies the original ensemble by a shifted version of itself. Then it double averages to compute the average over all samples (time average) and all rows (ensemble average).

```
function rx = ensemble_autocorr(ensemble, max_lag, t_start)
    if nargin < 3 || isempty(t_start), t_start = 1; end
    sub = ensemble(:, t_start:end);
    [~, n_samples] = size(sub);
    rx = zeros(1, 2*max_lag+1);
    for k = -max_lag:max_lag
        if k >= 0
            product = sub(:, 1:n_samples-k) .* sub(:, k+1:n_samples);
```

```
        else
            product = sub(:, -k+1:n_samples) .* sub(:, 1:n_samples+k);
        end
        N = size(product, 1); % Number of Realizations
        rx(k+max_lag+1) = sum( sum(product, 2)/size(product,2) ) / N;
    end
end
```

Then function `plot_ensemble_autocorr` takes a specific ensemble and it s line code name and a vector of starting times and plots the ensemble auto correlation functions for each starting time and stack them vertically. If start_times vector was not given it uses a default set of [1, 101, 354].

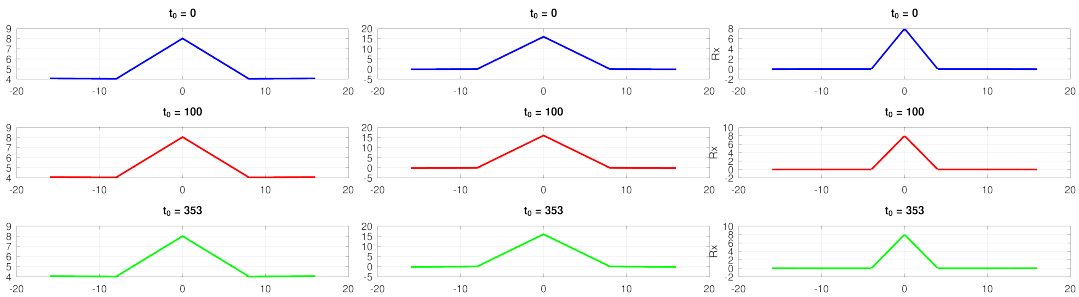
```
function plot_ensemble_autocorr(ensemble, name , max_lag, start_times)
    % Set default values if arguments are missing
    if nargin < 3 || isempty(max_lag), max_lag = 16; end
    if nargin < 4 || isempty(start_times), start_times = [1, 101, 354];
end

    % Create the lag vector for the x-axis
    lags = -max_lag:max_lag;
    num_starts = length(start_times);
    fig = figure('Name', name);
    % Pre-define a color palette to cycle through
    colors = {'b', 'r', 'g', 'm', 'c', 'k'};

    for i = 1:num_starts
        t_start = start_times(i);
        % Calculate autocorrelation anchored at this specific starting
time
        rx = ensemble_autocorr(ensemble, max_lag, t_start);
        % Create subplot stacked vertically
        subplot(num_starts, 1, i);
        % Pick a color, looping back to the start if num_starts >
length(colors)
        c_idx = mod(i - 1, length(colors)) + 1;
        % Plot the data
        plot(lags, rx, 'Color', colors{c_idx}, 'LineWidth', 2);
        ylabel('R_x', 'FontWeight', 'bold');
        title(sprintf('Starting Time t_0 = %d', t_start - 1));
        grid on;
        % Only add the X-label to the very bottom subplot for cleaner
visuals
        if i == num_starts
            xlabel('Lag', 'FontWeight', 'bold');
        end
    end
```

we then run the following code to see the results.

```
plot_ensemble_autocorr(ensembles_5h{1}, "Unipolar NRZ");
plot_ensemble_autocorr(ensembles_5h{2}, "Polar NRZ");
plot_ensemble_autocorr(ensembles_5h{3}, "Polar RZ");
```



a) Unipolar NRZ Autocorrelation ($R_x(0) = 8$, Roll-off= 4@8)

b) Polar NRZ Autocorrelation ($R_x(0) = 16$, Roll-off= 0@8)

c) Polar RZ Autocorrelation ($R_x(0) = 8$, Roll-off= 0@4)

Figure 5: Statistical Autocorrelation of each line-encoding for different start times.

In Figure 5, the autocorrelation function is calculated for varying start times over varying lags. It can be seen that $R_x(t, \tau)$ does not depend on the initial time and only depends on the time difference ($R_x(t, \tau) = R_x(\tau)$). Additionally, since m_x was calculated to be a constant over time, the random process is **Wide Sense Stationary (WSS)**. That can also be verified by observing how the autocorrelation is an even function.

4.3 The Time Mean and Autocorrelation of One Waveform

For a single realization $x_i(t)$, the time mean and autocorrelation both operate on a single 1×800 row of the ensemble matrix, producing a scalar mean and a $1 \times (\tau_{\max} + 1)$ autocorrelation vector respectively.

$$\langle x(t) \rangle = \overline{x}_i = \frac{1}{L_s} \sum_{t=1}^{L_s} x_i(t)$$

$$\langle x(t) \cdot x(t+\tau) \rangle = \frac{1}{L_s - \tau} \sum_{t=1}^{L_s - \tau} x_i(t) \cdot x_i(t+\tau) \quad [3]$$

the `time_autocorr` function was implemented through the following code

```
function rx_time = time_autocorr(waveform, max_lag)
  n = length(waveform);
  rx_time = zeros(1, 2*max_lag + 1);
  for k = -max_lag:max_lag
    % Define the overlapping segments
    if k >= 0
        seg1 = waveform(1:n-k);
        seg2 = waveform(k+1:n);
    else
        seg1 = waveform(-k+1:n);
        seg2 = waveform(1:n+k);
    end
    rx_time(k+max_lag+1) = sum(seg1 .* seg2) / length(seg1);
```

```
    end
end
```

For each lag value k, we multiply the corresponding elements in the overlapping area. We then calculate the mean by summing these products and dividing by the length of the overlap. This overlapping area is represented by seg1 and seg2, whose starting and ending indices are determined based on whether k is positive or negative, as illustrated in Figure 6.

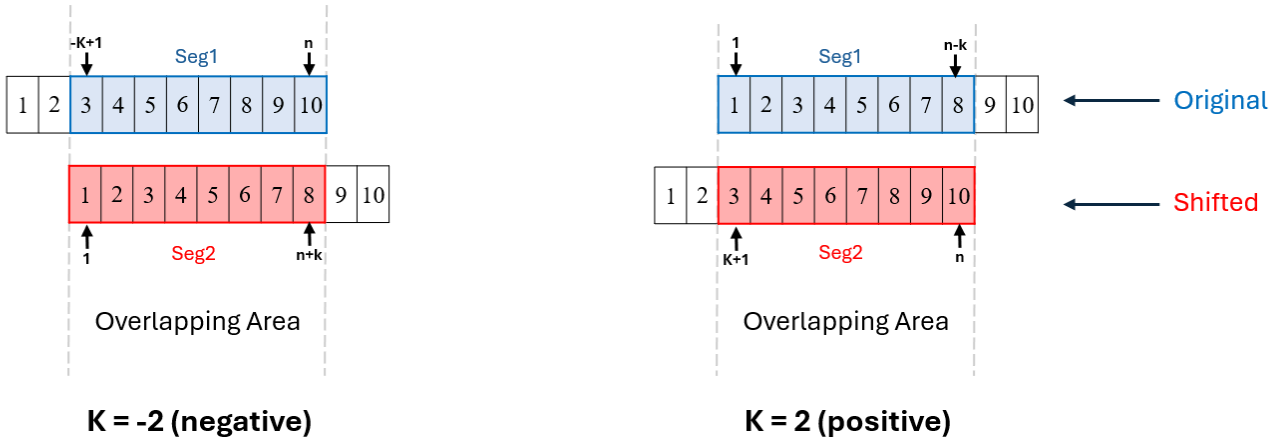


Figure 6: Visualization of the overlapping area and the starting and ending indices of `seg1` and `seg2` when $k < 0$ and $k \geq 0$.

To maintain a clean function-based script the function `plot_time_autocorrelations` was implemented. the function takes a cell array of ensembles calculate the time_autocorr of the first realization of each one then plots them on top of each other for Comparison. Additionally, it calculates the mean of the first waveform and prints them in the top left.

```
function plot_time_autocorrelations(ensembles)
    figure; hold on; grid on;
    % Ordered: Unipolar NRZ, Polar NRZ, Polar RZ
    names = {'Unipolar NRZ', 'Polar NRZ', 'Polar RZ'};
    colors = {'k', 'b', 'r'};

    for i = 1:3
        wv = ensembles{i}(1,:);
        plot(-32:32, time_autocorr(wv, 32), ...
            'Color', colors{i}, 'LineWidth', 1.5);
        stats_text = sprintf('%s: \\mu = %.4f', names{i}, mean(wv));
        text(0.02, 0.98 - (i-1)*0.06, stats_text, ...
            'Units', 'normalized', ...
            'Color', colors{i}, ...
            'FontSize', 10, ...
            'FontWeight', 'bold', ...
            'VerticalAlignment', 'top');
    end
```

```
    ylabel('Autocorrelation Rx'); legend(names); hold off;
end
```

we can then run the following code to see the results.

```
%default ensembles
ensembles_5h = generate_all_ensembles(control_flags);
% new ensembles with n_waveforms = 500 (default), n_bit = 100kb
ensembles_5h_100kb = generate_all_ensembles(control_flags, [], 1e5);

%Ploting
plot_time_autocorrelations(ensembles_5h);
plot_time_autocorrelations(ensembles_5h_100kb);
```

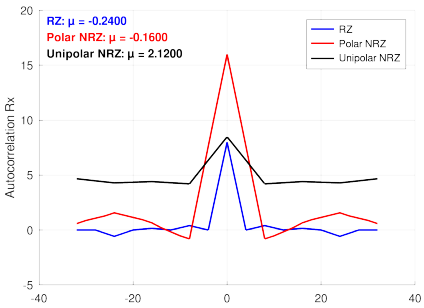


Figure 7: Mean and autocorrelation of one [100-bit] waveform.

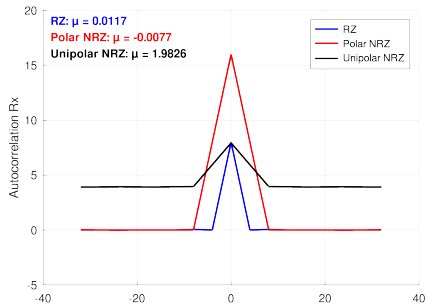


Figure 8: Mean and autocorrelation of a 10^5-bit waveform.

While similar, the time-based autocorrelation differs from the ensemble R_x in Figure 5.

However, increasing the number of bits makes it converge to the ensemble R_x. Figure 8 shows the calculation for the mean and autocorrelation of a waveform generated with 100 kilobits.

This characterizes the random process as **Ergodic**, meaning that the process statistics can be characterized through a single sufficiently long observation window.

4.4 Bandwidth of the Transmitted Signal

As established in section 4.2, the considered processes are wide-sense stationary (WSS). Consequently, their power spectral density (PSD) is defined as the Fourier transform of the autocorrelation function.

$$S_X(f) = \mathcal{F}.\mathcal{T}.\{R_X(\tau)\} \quad [4]$$

In practice, the autocorrelation function is available only for a finite number of discrete lags, sampled with a period T_S. Therefore, the continuous Fourier transform is approximated using the discrete Fourier transform (DFT), which is efficiently computed via the Fast Fourier Transform (FFT). The PSD is computed for each line coding scheme using this approach, and the frequency axis is scaled according to the sampling period $T_S = 10ms$ The resulting spectra are shown in Figure 9.

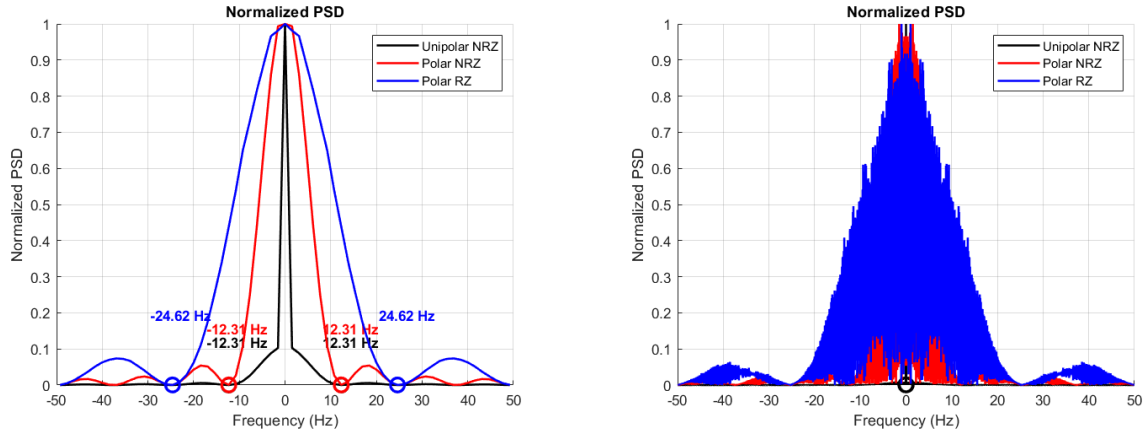


Figure 9: Normalized PSD for each line code. The spectrum is obtained via FFT of the autocorrelation function (R_x) limited to ± 32 lags (Left), ± 788 (Right)

The left plot corresponds to an autocorrelation window of ±32 lags, while the right plot uses a much larger window (±788 lags). It can be observed that the PSD obtained using the shorter window is smoother and more stable, whereas the longer window introduces noticeable fluctuations.

This behavior is due to the reduced reliability of autocorrelation estimates at larger lags, where fewer samples contribute to the averaging process, resulting in increased noise in the PSD. Therefore, limiting the autocorrelation to ±32 lags provides a good balance between capturing the essential spectral characteristics and maintaining numerical stability.

The bandwidth is defined as the frequency of the first spectral null. In practice, due to finite-length autocorrelation and numerical estimation effects, the PSD does not reach an exact zero at the null locations. Therefore, the first null is identified as the first local minimum after the main lobe, which provides a reliable estimate of the baseband bandwidth as shown in Figure 9 (Left).

Analytically, the $S_x(f)$ is given as a $\text{sinc}^2()$ function for each encoding since it is the transform of a convolved $\text{rect}()$. Accordingly, the first zero is given by $\frac{1}{T_p}$ where $T_p = 80\text{ms}$ is the pulse width. The discrepancy in the RZ encoding happens since it only transmits data for half the pulse. Effectively having a T_p of 40ms and double the bandwidth of NRZ encodings.

Table 1: Comparison of Analytical vs. Simulation Baseband Bandwidth

Encoding	**Effective T_p**	**Analytical BW ($\frac{1}{T_p}$)**	**Simulation BW**
Unipolar NRZ	80 ms	12.5 Hz	12.31 Hz
Polar NRZ	80 ms	12.5 Hz	12.31 Hz
Polar RZ	40 ms	25 Hz	24.62 Hz

```
%%
==========================================================================
% Cairo University | Faculty of Engineering
% Introduction to Communication Systems
% Project 2
% Team 11
% Submitted to Eng. Mohammed Khaled
%
% Description:
% This script generates, visualizes, and analyzes WSS random
processes
% using Unipolar NRZ, Polar NRZ, and Polar RZ line codes.
%
% NOTE: Please run this script section by section (using 'Run
Section')
%       to avoid triggering a fountain of plots all at once.
%
==========================================================================

clear; clc; close all;

% Control Flags (Defaults)
control_flags.A = 4;
control_flags.sample_per_bit = 8;
control_flags.sample_period = 10e-3;
control_flags.n_bits = 100;
control_flags.n_waveforms = 500;

% === Execution ===

%% Generate Ensembles
ensembles_5h = generate_all_ensembles(control_flags);

%% Plot waveforms

plot_sample_waveforms(ensembles_5h, control_flags);

%% Plot means
plot_means(ensembles_5h);

```

```
%% Ensemble auto correlations
plot_ensemble_autocorr(ensembles_5h{1}, "Unipolar NRZ");
plot_ensemble_autocorr(ensembles_5h{2}, "Polar NRZ");
plot_ensemble_autocorr(ensembles_5h{3}, "Polar RZ");

%% Plot Time auto correlation
plot_time_autocorrelations(ensembles_5h);

%% Plot PSD
plot_psd_from_ensembles(ensembles_5h, control_flags);

% ==== Excution End ====

%% Functions

% Ensemble Generation
function ensembles = generate_all_ensembles(control_flags,
n_waveforms, n_bits, A, spb)
    % Pass along optional arguments (empty brackets will trigger
defaults in generate_ensemble)
    if nargin < 5, spb = []; end
    if nargin < 4, A = []; end
    if nargin < 3, n_bits = []; end
    if nargin < 2, n_waveforms = []; end

    % Generates ensembles in order: Unipolar, Polar NRZ, Polar RZ
    ens_uz = generate_ensemble(@unipolar_nrz, control_flags,
n_waveforms, n_bits, A, spb);
    ens_pz = generate_ensemble(@polar_nrz, control_flags, n_waveforms,
n_bits, A, spb);
    ens_rz = generate_ensemble(@polar_rz, control_flags, n_waveforms,
n_bits, A, spb);

    ensembles = {ens_uz, ens_pz, ens_rz};
end
function ensemble = generate_ensemble(line_code_func, control_flags,
n_waveforms, n_bits, A, spb)
    % Use provided arguments or fall back to control_flags struct
    if nargin < 6 || isempty(spb), spb = control_flags.sample_per_bit;
end
    if nargin < 5 || isempty(A), A = control_flags.A; end
    if nargin < 4 || isempty(n_bits), n_bits = control_flags.n_bits;
end
    if nargin < 3 || isempty(n_waveforms), n_waveforms =
control_flags.n_waveforms; end

    % Direct allocation for memory efficiency
    waveform_length = n_bits * spb;
    ensemble = zeros(n_waveforms, waveform_length);

    for i = 1:n_waveforms
```

```
        random_bit_stream = randi([0 1], 1, n_bits);
        raw_wf = line_code_func(random_bit_stream, A, spb);

        % Apply random phase by circular shift and store directly in
matrix
        phase = randi([0, spb-1]);
        ensemble(i, :) = circshift(raw_wf, [0, -phase]);
    end
end

% Line Coding Functions
function waveform = unipolar_nrz(bits, A, spb)
    levels = bits * A;
    waveform = reshape(repmat(levels(:)', spb, 1), 1, []);
end
function waveform = polar_nrz(bits, A, spb)
    levels = (2*bits(:)' - 1) * A;
    waveform = reshape(repmat(levels, spb, 1), 1, []);
end
function waveform = polar_rz(bits, A, spb)
    half_spb = floor(spb / 2);
    levels = (2*bits(:)' - 1) * A;
    raw_matrix = repmat(levels, spb, 1);

    % Zero out the second half of each bit period to make it Return-
to-Zero
    raw_matrix(half_spb+1:end, :) = 0;

    waveform = reshape(raw_matrix, 1, []);
end

% Analysis Functions
function rx = ensemble_autocorr(ensemble, max_lag, t_start)
    if nargin < 3 || isempty(t_start), t_start = 1; end
    sub = ensemble(:, t_start:end);
    [~, n_samples] = size(sub);
    rx = zeros(1, 2*max_lag+1);
    for k = -max_lag:max_lag
        if k >= 0
            product = sub(:, 1:n_samples-k) .* sub(:, k+1:n_samples);
        else
            product = sub(:, -k+1:n_samples) .* sub(:, 1:n_samples+k);
        end
        N = size(product, 1); % Number of Realizations
        rx(k+max_lag+1) = sum( sum(product, 2)/size(product,2) ) / N;
    end
end
function rx_time = time_autocorr(waveform, max_lag)
    n = length(waveform);
    rx_time = zeros(1, 2*max_lag + 1);
    for k = -max_lag:max_lag
```

```
        % Define the overlapping segments
        if k >= 0
            seg1 = waveform(1:n-k);
            seg2 = waveform(k+1:n);
        else
            seg1 = waveform(-k+1:n);
            seg2 = waveform(1:n+k);
        end
        rx_time(k+max_lag+1) = sum(seg1 .* seg2) / length(seg1);
    end
end


% Visualization Functions

function plot_sample_waveforms(ensembles, control_flags, n_show,
samples_to_show)
    % Default arguments
    if nargin < 3 || isempty(n_show), n_show = 5; end
    if nargin < 4 || isempty(samples_to_show), samples_to_show = 240;
end

    names = {'Unipolar NRZ', 'Polar NRZ', 'Polar RZ'};
    num_line_codes = length(ensembles);

    % Create a large figure to fit the grid
    figure('Name', 'Sample Waveforms');

    for lc_idx = 1:num_line_codes
        current_ens = ensembles{lc_idx};
        % Ensure we don't try to show more waveforms/samples than
exist
        n = min(n_show, size(current_ens, 1));
        max_samples = min(samples_to_show, size(current_ens, 2));

        y_limits = [-control_flags.A-1, control_flags.A+1]; % Find y-
limits

        for wf_idx = 1:n
            subplot_idx = (wf_idx - 1) * num_line_codes + lc_idx;
            subplot(n, num_line_codes, subplot_idx);
            % Plot the specific waveform slice
            plot(current_ens(wf_idx, 1:max_samples), ...
                'LineWidth', 1.5, ...
                'Color', '#0072BD');
            ylim(y_limits);
            grid on;
            % Add column titles only on the top row
            if wf_idx == 1
                title(names{lc_idx} ...
                    , 'FontWeight', 'bold', ...
```

```
                        'FontSize', 11);
            end
            % Add X-axis labels only on the bottom row
            if wf_idx == n
                xlabel('Sample Index');
            end
        end
    end
end
function plot_means(ensembles)
    % Ordered: Unipolar NRZ, Polar NRZ, Polar RZ
    names = {'Unipolar NRZ', 'Polar NRZ', 'Polar RZ'};
    colors = {'k', 'r', 'b'};
    n_waveforms = size(ensembles{1}, 1);

    figure; hold on;
    for i = 1:length(ensembles)
        current_ens = ensembles{i};
        ens_mean = sum(current_ens, 1)/size(current_ens, 1);

        average = sum(ens_mean)/length(ens_mean);
        variance = var(ens_mean);

        plot(ens_mean, 'Color', colors{i}, 'LineWidth', 1.5,
'DisplayName', names{i});
        stats_text = sprintf('%s: \\mu = %.4f, \\sigma^2 = %.4e',
names{i}, average, variance);
        text(0.02, 0.98 - (i-1)*0.06, stats_text, 'Units',
'normalized', ...
            'Color', colors{i}, 'FontSize', 10, 'FontWeight', 'bold',
'VerticalAlignment', 'top');
    end
    ylim([-1, 4]); xlabel('Samples'); ylabel('Ensemble Mean
Amplitude');
    title(sprintf('Ensemble Means and Variance (%d waveforms)',
n_waveforms));
    legend('Location', 'northeast'); grid on; hold off;
end
function plot_time_autocorrelations(ensembles)
    figure; hold on; grid on;
    % Ordered: Unipolar NRZ, Polar NRZ, Polar RZ
    names = {'Unipolar NRZ', 'Polar NRZ', 'Polar RZ'};
    colors = {'k', 'r', 'b'};

    for i = 1:3
        wv = ensembles{i}(1,:);
        plot(-32:32, time_autocorr(wv, 32), ...
            'Color', colors{i}, 'LineWidth', 1.5);
        stats_text = sprintf('%s: \\mu = %.4f', names{i}, sum(wv)/
length(wv));
        text(0.02, 0.98 - (i-1)*0.06, stats_text, ...
```

```
            'Units', 'normalized', ...
            'Color', colors{i}, ...
            'FontSize', 10, ...
            'FontWeight', 'bold', ...
            'VerticalAlignment', 'top');
    end
    ylabel('Autocorrelation Rx'); legend(names); hold off;
end
function plot_ensemble_autocorr(ensemble, name , max_lag, start_times)
    % Set default values if arguments are missing
    if nargin < 3 || isempty(max_lag), max_lag = 16; end
    if nargin < 4 || isempty(start_times), start_times = [1, 101,
354]; end

    % Create the lag vector for the x-axis
    lags = -max_lag:max_lag;
    num_starts = length(start_times);

    % Initialize the figure window and automatically center it
    fig = figure('Name', name);

    % Pre-define a color palette to cycle through
    colors = {'b', 'r', 'g', 'm', 'c', 'k'};

    for i = 1:num_starts
        t_start = start_times(i);

        % Calculate autocorrelation anchored at this specific starting
time
        rx = ensemble_autocorr(ensemble, max_lag, t_start);

        % Create subplot stacked vertically
        subplot(num_starts, 1, i);

        % Pick a color, looping back to the start if num_starts >
length(colors)
        c_idx = mod(i - 1, length(colors)) + 1;

        % Plot the data
        plot(lags, rx, 'Color', colors{c_idx}, 'LineWidth', 2);
        ylabel('R_x', 'FontWeight', 'bold');
        title(sprintf('Starting Time t_0 = %d', t_start - 1));
        grid on;

        % Only add the X-label to the very bottom subplot for cleaner
visuals
        if i == num_starts
            xlabel('Lag', 'FontWeight', 'bold');
        end
    end
end
```

```

function plot_psd_from_ensembles(ensembles, control_flags,
sample_period)
    if nargin < 3 || isempty(sample_period)
        sample_period = control_flags.sample_period;
    end
    max_lag = 32;
    N = 2 * max_lag + 1;
    fs = 1 / sample_period;
    freq_axis = (-(N-1)/2:(N-1)/2) * (fs / N);

    % Ordered: Unipolar NRZ, Polar NRZ, Polar RZ
    names = {'Unipolar NRZ', 'Polar NRZ', 'Polar RZ'};
    colors = {'k', 'r', 'b'};

    figure; hold on; grid on;
    p = zeros(1, 3);
    for i = 1:3
        rxs = ensemble_autocorr(ensembles{i}, max_lag);
        PSD = abs(fft(ifftshift(rxs)));
        PSD = PSD / max(PSD);
        PSD0 = fftshift(PSD);
        p(i) = plot(freq_axis, PSD0, ...
            'Color', colors{i}, 'LineWidth', 1.5);

        % Find all local minima (peaks of the negative PSD)
        % with max threshold 0.2
        [~, all_nulls] = findpeaks(-PSD0, MinPeakHeight=-0.2);

        % Identify nulls closest to the center (DC) frequency
        center_idx   = (N + 1) / 2;
        idx_before   = all_nulls(find(all_nulls < center_idx, 1,
'last'));
        idx_after    = all_nulls(find(all_nulls > center_idx, 1,
'first'));
        ind = [idx_before, idx_after];

        labels = cellstr(num2str(freq_axis(ind)', '%.2f Hz'));
        plot(freq_axis(ind), PSD0(ind), 'ro',...
            'Color', colors{i}, ...
            'MarkerSize', 10, 'LineWidth', 2);

        text(-5 + freq_axis(ind), i * 3.8e-2 + 0.08 + PSD0(ind), ...
            labels, 'Color', colors{i}, ...
            'FontSize', 10, ...
            'FontWeight', 'bold');
    end
    legend(p, names);
    xlabel('Frequency (Hz)');
    ylabel('Normalized PSD');
    title('Normalized PSD');
```

```
end
```

Bibliography

[1] Dr. Mohammed Nafi, “Lecture 4, Slide 6.” [Online]. Available: https://drive.google.com/drive/folders/1aRjPvyJMmmfu2QyYCgkd-onW8zcEnJcH

[2] Dr. Mohammed Nafi, “Lecture 4, Slide 7.” [Online]. Available: https://drive.google.com/drive/folders/1aRjPvyJMmmfu2QyYCgkd-onW8zcEnJcH

[3] Dr. Mohammed Nafi, “Lecture 4, Slide 23.” [Online]. Available: https://drive.google.com/drive/folders/1aRjPvyJMmmfu2QyYCgkd-onW8zcEnJcH

[4] Dr. Mohammed Nafi, “Lecture 4, Slide 16.” [Online]. Available: https://drive.google.com/drive/folders/1aRjPvyJMmmfu2QyYCgkd-onW8zcEnJcH